

pgBackRest

Guide Complet de Sauvegarde PostgreSQL

Installation • Configuration • Sauvegarde • Restauration • PITR

1. Comprendre pgBackRest

1.1 Qu'est-ce que pgBackRest ?

pgBackRest est un outil open source de sauvegarde et restauration fiable pour PostgreSQL. Il est conçu pour gérer des bases de données de toutes tailles, des plus petites aux plus grandes, avec des fonctionnalités avancées de compression, chiffrement, parallélisme et archivage WAL.

Contrairement à pg_basebackup (outil natif de PostgreSQL), pgBackRest offre une gestion complète du cycle de vie des sauvegardes : planification, rétention, chiffrement, stockage distant (S3, Azure, GCS) et restauration point-in-time.

1.2 Concepts fondamentaux

Le Stanza

Un stanza est l'unité de configuration de pgBackRest pour un cluster PostgreSQL. Il définit où se trouve la base, comment elle est sauvegardée, et les options d'archivage. Chaque cluster PostgreSQL correspond à un stanza.

BON A SAVOIR

Choisissez un nom de stanza qui décrit la fonction du cluster (ex: "production", "app", "dw") plutôt que son nom technique. Ce nom sera utilisé pour les sauvegardes et les restaurations.

Le Repository

Le repository est l'espace de stockage où pgBackRest conserve les sauvegardes et les archives WAL. Il peut être :

- Un répertoire local sur le serveur PostgreSQL
- Un serveur de sauvegarde dédié (via SSH ou TLS)
- Un stockage objet cloud : AWS S3, Azure Blob Storage, Google Cloud Storage
- Plusieurs repositories simultanément (jusqu'à 4) pour la redondance

Le WAL et pgBackRest

pgBackRest s'appuie entièrement sur le mécanisme WAL de PostgreSQL. Les commandes archive-push et archive-get remplacent l'archive_command natif pour gérer l'archivage de manière fiable, compressée et chiffrée.

1.3 Architecture générale

Composant	Rôle
Serveur PostgreSQL (pg-primary)	Exécute la base, génère les WAL, déclenche les backups
pgBackRest (client)	Outil installé sur le serveur PG, gère archive-push/get
Repository	Stockage des sauvegardes et archives WAL
pgbackrest.conf	Fichier de configuration central (/etc/pgbackrest/)
Stanza	Configuration spécifique à un cluster PostgreSQL

2. Types de Sauvegarde

2.1 Sauvegarde Complète (Full)

Une sauvegarde full copie l'intégralité du répertoire de données PostgreSQL. C'est la première sauvegarde obligatoire et la seule que pgBackRest peut restaurer de manière indépendante, sans dépendre d'une autre sauvegarde.

- Première sauvegarde d'un cluster : toujours une full
- Indépendante : ne dépend d'aucun autre fichier
- La plus volumineuse en espace disque

- Sert de référence pour les sauvegardes différentielles et incrémentales

2.2 Sauvegarde Différentielle (Diff)

Une sauvegarde différentielle copie uniquement les fichiers modifiés depuis la dernière sauvegarde full. Pour la restaurer, pgBackRest a besoin de la full de référence ET de la diff.

- Base de référence : toujours la dernière sauvegarde full
- Plus petite qu'une full, mais grossit au fil du temps dans la semaine
- Restauration : full + diff (2 éléments)
- RTO (temps de restauration) meilleur que l'incrémentale car moins de dépendances

2.3 Sauvegarde Incrémentale (Incr)

Une sauvegarde incrémentale copie uniquement les fichiers modifiés depuis la dernière sauvegarde (qu'elle soit full, diff ou incr). C'est la plus petite en taille mais celle qui nécessite le plus de dépendances pour la restauration.

- Base de référence : la sauvegarde précédente (quelle que soit son type)
- La plus petite en espace disque
- Restauration : toute la chaîne full + diff + toutes les incr intermédiaires
- RTO plus long car plus de fichiers à assembler

2.4 Comparaison des types de sauvegarde

Critère	Full	Différentielle	Incrémentale
Taille	Très grande	Moyenne (croît)	Petite
Dépendances	Aucune	Full seule	Full + Diff + toutes Incr
Restauration (RTO)	Rapide	Rapide	Plus long
Fréquence recommandée	Hebdomadaire	Quotidienne	Plusieurs/jour
Commande	--type=full	--type=diff	--type=incr

PRINCIPE CLÉ : WAL + Checkpoint + Archivage = PITR

Le PITR n'est possible que parce que le WAL enregistre chaque changement, que le Checkpoint ancre l'état sur disque, et que l'Archivage Continu préserve ces journaux dans le temps. pgBackRest orchestre cette chaîne de bout en bout.

3. Scénario de Sauvegarde Recommandé

3.1 Stratégie : Full + Diff + Incr + Archivage WAL

Le scénario suivant combine les trois types de sauvegarde avec l'archivage continu des WAL pour offrir le meilleur équilibre entre espace de stockage, RPO (perte de données maximale) et RTO (temps de restauration).

Jour	Type	Heure	Justification
Dimanche	Full	01h00	Référence hebdomadaire complète
Lundi	Différentielle	01h00	Capture tous les changements depuis dimanche
Mardi	Différentielle	01h00	Capture tous les changements depuis

			dimanche
Mercredi	Différentielle	01h00	Capture tous les changements depuis dimanche
Jeudi	Incrémentale	06h00 + 18h00	Complète la diff du matin, 2x/jour
Vendredi	Incrémentale	06h00 + 18h00	Complète la diff du matin, 2x/jour
Samedi	Incrémentale	06h00 + 18h00	Complète la diff du matin, 2x/jour
Continu	Archivage WAL	En temps réel	RPO = quelques minutes (archive_timeout)

3.2 Rétention recommandée

- 2 sauvegardes full conservées (2 semaines de couverture)
- Toutes les différentielles associées aux full retenues
- Toutes les incrémentales associées aux diff retenues
- Archives WAL conservées pour couvrir la durée des full (RPO complet)

4. Installation et Configuration (Step by Step)

Cette section guide l'installation complète de pgBackRest sur un serveur Ubuntu/Debian avec PostgreSQL 16.

PRÉREQUIS

PostgreSQL 16 installé et fonctionnel. Utilisateur postgres disponible. Accès sudo sur le serveur. Espace disque suffisant pour le repository.

Étape 1 : Installation de pgBackRest

Via le repository officiel PGDG (recommandé)

```
# Ajouter le repository PGDG si pas encore fait
sudo apt-get install -y curl ca-certificates
sudo install -d /usr/share/postgresql-common/pgdg
sudo curl -o /usr/share/postgresql-common/pgdg/apt.postgresql.org.asc \
  --fail https://www.postgresql.org/media/keys/ACCC4CF8.asc

# Ajouter le repository
sudo sh -c 'echo "deb [signed-by=/usr/share/postgresql-
common/pgdg/apt.postgresql.org.asc] \
https://apt.postgresql.org/pub/repos/apt $(lsb_release -cs)-pgdg main" > \
/etc/apt/sources.list.d/pgdg.list'

# Installer pgBackRest
sudo apt-get update
sudo apt-get install -y pgbackrest

# Verifier l'installation
pgbackrest version
```

Étape 2 : Création des répertoires et fichiers de configuration

```
# Creer les repertoires necessaires
sudo mkdir -p /var/log/pgbackrest
sudo chown postgres:postgres /var/log/pgbackrest
sudo chmod 750 /var/log/pgbackrest

# Creer le repertoire de configuration
sudo mkdir -p /etc/pgbackrest/conf.d

# Creer le fichier de configuration principal
```

```
sudo touch /etc/pgbackrest/pgbackrest.conf
sudo chmod 640 /etc/pgbackrest/pgbackrest.conf
sudo chown postgres:postgres /etc/pgbackrest/pgbackrest.conf

# Créer le repertoire du repository (stockage local)
sudo mkdir -p /var/lib/pgbackrest
sudo chmod 750 /var/lib/pgbackrest
sudo chown postgres:postgres /var/lib/pgbackrest
```

Étape 3 : Configuration de pgBackRest

Editer le fichier /etc/pgbackrest/pgbackrest.conf avec le contenu suivant :

```
# /etc/pgbackrest/pgbackrest.conf

[global]
# Chemin du repository de sauvegarde
repo1-path=/var/lib/pgbackrest

# Retention : conserver 2 full backups
repo1-retention-full=2

# Chiffrement (remplacer par une cle forte)
# repo1-cipher-type=aes-256-cbc
# repo1-cipher-pass=VOTRE_CLE_FORTE_ICI

# Logging
log-level-console=info
log-level-file=detail
log-path=/var/log/pgbackrest

# Performance : utiliser 2 processus paralleles
process-max=2

# Demarrage rapide du backup (checkpoint immediat)
start-fast=y

# Compression archive WAL (niveau reduit pour rapidite)
[global:archive-push]
compress-level=3

# Stanza pour le cluster PostgreSQL 16
[production]
pg1-path=/var/lib/postgresql/16/main
pg1-port=5432
```

Étape 4 : Configuration de PostgreSQL

Modifier /etc/postgresql/16/main/postgresql.conf :

```
# /etc/postgresql/16/main/postgresql.conf

# Niveau WAL minimum pour la replication et l'archivage
wal_level = replica

# Activer l'archivage WAL
archive_mode = on

# Commande d'archivage via pgBackRest
archive_command = 'pgbackrest --stanza=production archive-push %p'

# Forcer l'archivage d'un segment toutes les 5 min (RPO max = 5 min)
# archive_timeout = 300
```

```
# Activer les checksum de pages (recommande)  
# A activer lors de initdb uniquement : initdb --data-checksums
```

Redémarrer PostgreSQL pour appliquer les changements

```
sudo systemctl restart postgresql@16-main  
  
# Verifier que PostgreSQL est bien relance  
sudo systemctl status postgresql@16-main  
  
# Verifier le statut d'archivage  
sudo -u postgres psql -c "SELECT name, setting FROM pg_settings WHERE name  
LIKE 'archive%';"
```

Étape 5 : Création et vérification du Stanza

```
# Créer le stanza (initialiser le repository)  
sudo -u postgres pgbackrest --stanza=production --log-level-console=info  
stanza-create  
  
# Verifier que la configuration est correcte (archivage + backup)  
sudo -u postgres pgbackrest --stanza=production --log-level-console=info check
```

La commande check force un changement de segment WAL (pg_switch_wal()) et vérifie que le segment arrive bien dans le repository. Si elle réussit, la configuration est validée.

RÉSULTAT ATTENDU

```
P00 INFO: WAL segment 0000000100000000000000001 successfully archived to  
'/var/lib/pgbackrest/...' P00 INFO: check command end: completed successfully
```

5. Exécuter les Sauvegardes

5.1 Sauvegarde Full

```
# Lancer une sauvegarde full  
sudo -u postgres pgbackrest --stanza=production --type=full backup  
  
# Avec logs detaillés  
sudo -u postgres pgbackrest --stanza=production --type=full --log-level-  
console=info backup  
  
# Verifier le resultat  
sudo -u postgres pgbackrest --stanza=production info
```

5.2 Sauvegarde Différentielle

```
# Lancer une sauvegarde differentielle  
# (necessite une full prealable)  
sudo -u postgres pgbackrest --stanza=production --type=diff backup
```

5.3 Sauvegarde Incrémentale

```
# Lancer une sauvegarde incrementale  
# (necessite une full ou diff prealable)  
sudo -u postgres pgbackrest --stanza=production --type=incr backup
```

5.4 Planification avec Cron (scénario complet)

Ajouter les lignes suivantes dans la crontab de l'utilisateur postgres :

```
# Editer la crontab de postgres  
sudo -u postgres crontab -e
```

```
# --- CONTENU DE LA CRONTAB ---

# Sauvegarde FULL tous les dimanches a 01h00
0 1 * * 0 pgbackrest --stanza=production --type=full backup

# Sauvegarde DIFFERENTIELLE lundi-samedi a 01h00
0 1 * * 1-6 pgbackrest --stanza=production --type=diff backup

# Sauvegarde INCREMENTALE jeudi-samedi a 06h00 et 18h00
0 6,18 * * 4-6 pgbackrest --stanza=production --type=incr backup

# Expiration manuelle des backups obsoletes (optionnel, gere automatiquement)
# 0 2 * * 0 pgbackrest --stanza=production expire
```

5.5 Consulter les sauvegardes disponibles

```
# Afficher toutes les sauvegardes et archives WAL
sudo -u postgres pgbackrest --stanza=production info

# Format JSON (pour scripts de monitoring)
sudo -u postgres pgbackrest --stanza=production info --output=json
```

Exemple de sortie typique :

```
stanza: production
  status: ok
  cipher: none

  db (current)
    wal archive min/max (16):
00000001100000000000000001/00000001100000000000000009

    full backup: 20241201-010000F
      timestamp start/stop: 2024-12-01 01:00:00 / 2024-12-01 01:15:32
      wal start/stop: 00000001100000000000000002 /
00000001100000000000000003
      database size: 10.5GB, backup size: 10.5GB

    diff backup: 20241201-010000F_20241202-010000D
      timestamp start/stop: 2024-12-02 01:00:00 / 2024-12-02 01:03:12
      database size: 10.6GB, backup size: 512MB

    incr backup: 20241201-010000F_20241202-010000D_20241203-060000I
      timestamp start/stop: 2024-12-03 06:00:00 / 2024-12-03 06:01:05
      database size: 10.7GB, backup size: 128MB
```

6. Procédures de Restauration

ATTENTION

Toute restauration écrase le répertoire de données. Arrêtez toujours PostgreSQL avant de lancer une restauration et sauvegardez le répertoire existant si possible.

6.1 Restauration Complète (dernière sauvegarde)

Restaure la dernière sauvegarde disponible (full, diff ou incr) et rejoue tous les WAL disponibles jusqu'au moment le plus récent.

```
# 1. Arrêter PostgreSQL
sudo systemctl stop postgresql@16-main

# 2. Sauvegarder le repertoire de donnees existant (optionnel)
sudo mv /var/lib/postgresql/16/main /var/lib/postgresql/16/main.old
```

```
# 3. Recreer le repertoire vide
sudo mkdir -p /var/lib/postgresql/16/main
sudo chown postgres:postgres /var/lib/postgresql/16/main
sudo chmod 700 /var/lib/postgresql/16/main

# 4. Lancer la restauration (derniere sauvegarde)
sudo -u postgres pgbackrest --stanza=production restore

# 5. Demarrer PostgreSQL (il entre en mode recovery automatiquement)
sudo systemctl start postgresql@16-main

# 6. Suivre la progression du recovery
sudo tail -f /var/log/postgresql/postgresql-16-main.log

# 7. Verifier que la base est operationnelle
sudo -u postgres psql -c "SELECT now(), pg_is_in_recovery();"

```

6.2 Restauration à partir d'une sauvegarde spécifique

Pour restaurer à partir d'une sauvegarde précise (par exemple la full du dimanche sans rejouer les WAL), utiliser l'option `--set` :

```
# Lister les sauvegardes pour identifier le label
sudo -u postgres pgbackrest --stanza=production info

# Restaurer une sauvegarde specifique (ex: full du 01/12/2024)
sudo systemctl stop postgresql@16-main

sudo -u postgres pgbackrest --stanza=production \
  --set=20241201-010000F \
  restore

sudo systemctl start postgresql@16-main

```

6.3 Restauration PITR (Point-In-Time Recovery)

Le PITR permet de restaurer la base à un instant précis, par exemple juste avant une suppression accidentelle de données.

Scénario : une table a été supprimée accidentellement à 14h37

```
# 1. Arrêter PostgreSQL
sudo systemctl stop postgresql@16-main

# 2. Vider le repertoire de donnees
sudo -u postgres find /var/lib/postgresql/16/main -mindepth 1 -delete

# 3. Restauration PITR : cible 1 minute avant l'incident
sudo -u postgres pgbackrest --stanza=production \
  --type=time \
  "--target=2024-12-03 14:36:00" \
  --target-action=promote \
  restore

# 4. Demarrer PostgreSQL
sudo systemctl start postgresql@16-main

# 5. Verifier que la table existe de nouveau
sudo -u postgres psql -c "\dt ma_table_supprimee"

# 6. Si la restauration est validee, supprimer l'ancien repertoire
sudo rm -rf /var/lib/postgresql/16/main.old

```


6.4 Options de cible PITR

Type de cible	Paramètre	Exemple
Date et heure	--type=time --target="..."	"2024-12-03 14:36:00"
Point nommé	--type=name --target="..."	"avant_migration"
Identifiant transaction	--type=xid --target="..."	"12345678"
LSN (position WAL)	--type=lsn --target="..."	"0/3000020"
Fin de restauration	--type=immediate	(pas de --target)

6.5 Restauration Delta (partielle, sans tout écraser)

Le mode delta permet de ne restaurer que les fichiers manquants ou corrompus, sans toucher aux fichiers intacts. Très utile pour les grandes bases de données.

```
# Restauration delta : ne restaure que les fichiers différents
# Beaucoup plus rapide sur les grandes bases
sudo systemctl stop postgresql@16-main

sudo -u postgres pgbackrest --stanza=production \
  --delta \
  restore

sudo systemctl start postgresql@16-main
```

6.6 Restauration d'une base spécifique uniquement

pgBackRest peut restaurer uniquement certaines bases de données d'une instance (les autres bases seront en mode lecture seule jusqu'à la fin du recovery).

```
# Restaurer uniquement la base "ma_base"
sudo -u postgres pgbackrest --stanza=production \
  --db-include=ma_base \
  restore
```

7. Surveillance et Validation

7.1 Surveiller l'archivage WAL

```
# Verifier le statut de l'archivage PostgreSQL
sudo -u postgres psql -c "SELECT * FROM pg_stat_archiver;"

# Colonnes importantes :
# archived_count      : nombre de WAL archives avec succes
# failed_count        : nombre d'echecs (doit etre 0)
# last_archived_wal    : dernier WAL archive avec succes
# last_failed_wal      : dernier WAL en echec (doit etre vide)

# Verifier l'espace du repertoire WAL
sudo du -sh /var/lib/postgresql/16/main/pg_wal/

# Verifier l'espace du repository pgBackRest
sudo du -sh /var/lib/pgbackrest/
```

7.2 Valider l'intégrité des sauvegardes

```
# Verifier la configuration pgBackRest
sudo -u postgres pgbackrest --stanza=production check

# Verifier l'integrite du repository (checksums)
sudo -u postgres pgbackrest --stanza=production verify
```

```
# Afficher l'info detaillee de toutes les sauvegardes
sudo -u postgres pgbackrest --stanza=production info
```

7.3 Checklist de surveillance

Point à surveiller	Commande / Vue	Fréquence
Archivage WAL actif	pg_stat_archiver (failed_count = 0)	Quotidien
Espace disque pg_wal/	du -sh pg_wal/	Quotidien
Espace repository	du -sh /var/lib/pgbackrest/	Hebdomadaire
Succes derniere sauvegarde	pgbackrest info	Après chaque backup
Test de restauration	Environnement de test	Mensuel
Logs pgBackRest	/var/log/pgbackrest/	Après incident

8. Référence des Paramètres Clés

8.1 Paramètres globaux pgbackrest.conf

Paramètre	Défaut	Description
repo1-path	(requis)	Chemin du repository de sauvegarde
repo1-retention-full	(requis)	Nombre de full backups à conserver
repo1-retention-diff	(optionnel)	Nombre de diff backups à conserver
repo1-cipher-type	none	Type de chiffrement (aes-256-cbc)
repo1-cipher-pass	(requis si chiffré)	Clé de chiffrement (32+ caractères)
process-max	1	Processus parallèles (recomm: nb CPUs / 2)
start-fast	n	Checkpoint immédiat avant backup (y recommandé)
log-level-console	warn	Niveau de log console (info en prod)
log-level-file	info	Niveau de log fichier (detail pour debug)
compress-type	gz	Compression : gz, bz2, lz4, zst
delta	n	Activer le mode delta par défaut
archive-timeout	60s	Délai max pour archiver un WAL

8.2 Paramètres PostgreSQL liés à pgBackRest

Paramètre	Valeur requise	Raison
wal_level	replica ou logical	Requis pour archivage et réplication
archive_mode	on	Active l'archivage des WAL
archive_command	pgbackrest archive-push %p	Commande d'archivage pgBackRest
archive_timeout	300 (optionnel)	RPO max = 5 min sur bases peu actives
fsync	on	Ne jamais désactiver en production
full_page_writes	on	Protection contre écritures

partielles

9. Synthèse et Points Clés

9.1 La chaîne indissociable

Mécanisme	Rôle	Sans lui...
WAL	Enregistre chaque transaction	Aucune reconstruction possible
Checkpoint	Ancre l'état sur disque, recycle les WAL	Les WAL s'accumulent indéfiniment
Archivage Continu	Préserve les WAL dans le temps	Fenêtre de PITR limitée au dernier checkpoint
pgBackRest	Orchestre l'ensemble + chiffrement + rétention	Gestion manuelle complexe et fragile
PITR	Exploite les WAL archivés pour voyager dans le temps	Restauration seulement à la dernière sauvegarde complète

9.2 Bonnes pratiques essentielles

- Toujours tester l'archivage (pgbackrest check) avant la première sauvegarde
- Ne jamais écraser un fichier WAL archivé existant
- Surveiller pg_stat_archiver : failed_count doit toujours être à 0
- Prévoir de l'espace supplémentaire pour pg_wal/ (max_wal_size non strict)
- Tester la restauration régulièrement sur un environnement de test
- Activer le chiffrement (repo1-cipher-type=aes-256-cbc) pour les données sensibles
- Conserver au minimum 2 full backups pour garantir une fenêtre de récupération
- Utiliser process-max > 1 pour accélérer les backups et restaurations

RAPPEL FONDAMENTAL

WAL enregistre. Checkpoint ancre. L'Archivage Continu préserve. Le PITR restaure. Quatre mécanismes, une seule chaîne — briser un seul maillon, c'est perdre la capacité de récupérer.